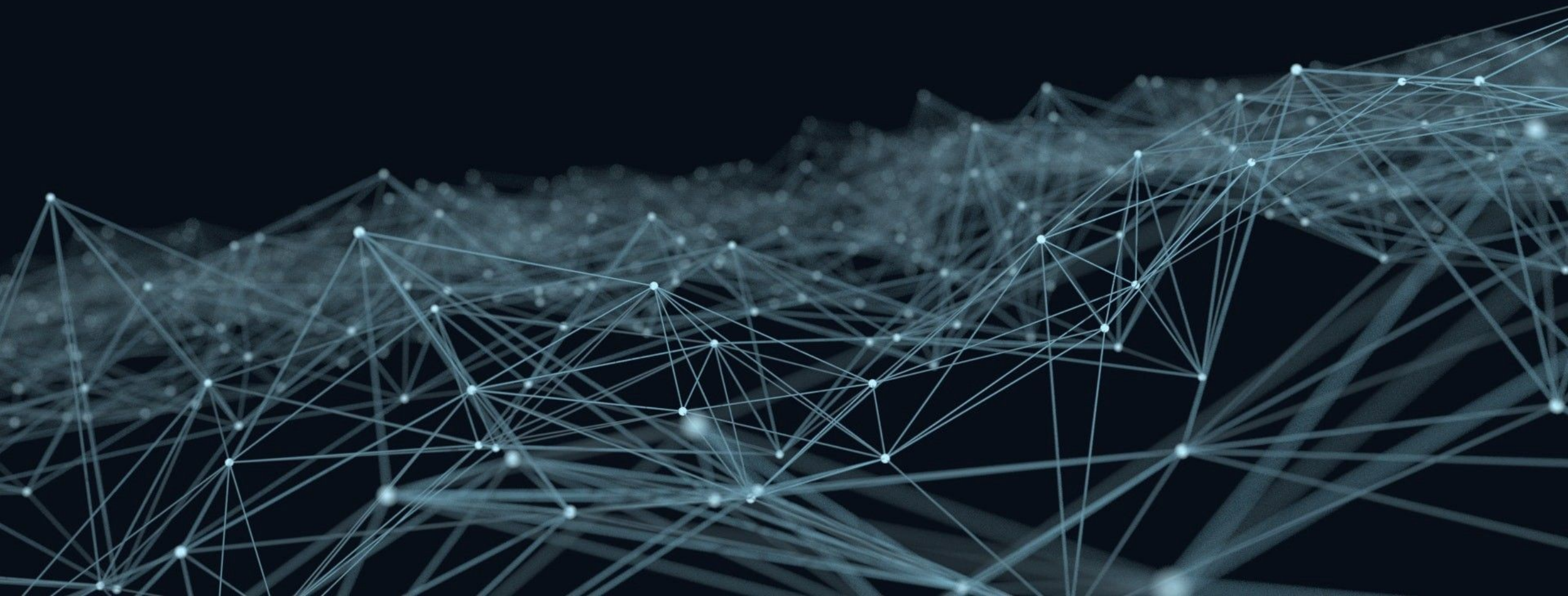
Java Full Stack Development

1. Java 17-21 Advanced Features



Module Topics

- Java LTS release schedules
- Records as immutable data carriers
- Sealed classes for closed type hierarchies
- Pattern matching
- Multi-line text blocks
- Virtual threads

Java Releases

The LTS Release Model

- Since Java 9
  - Oracle has shipped a new Java version every six months.
  - This cadence is too fast for most enterprises to track
  - Long-Term Support (LTS) releases
    - Receive commercial and community patches for multi-year window
    - Intended as safe production targets
  - Never target a non-LTS release for production services
    - Specifically versions 18, 19, 20, 22, 23, 24
    - Preview features can break between releases

Release	Type	GA Date	Relevance
Java 8	LTS	March 2014	Legacy — still widespread; lacks modern language features
Java 11	LTS	September 2018	Widely deployed; HttpClient, var keyword
Java 17	LTS	September 2021	Current minimum target — all features in this module are stable
Java 21	LTS	September 2023	Preferred new target — Virtual Threads (GA), pattern matching final
Java 25	LTS	September 2025	Future — begin planning migration

Java 17 versus Java 21

- New projects should target Java 21
  - It is LTS, Spring Boot 3.2+ is tuned for it
  - Virtual Threads are generally available and the API locked for production use
- For projects already on Java 17
  - Stay until there is a clear migration path
  - Java 17 is fully supported until at least 2029
- For this class we will primarily focus on Java 17 features
  - However, we will preview some of the Java 21 features not in Java 17

Java 17 vs Java 21

Feature	Java 17	Java 21
Sealed Classes	Introduced sealed classes, allowing more control over class inheritance.	No changes; already present in Java 17.
Pattern Matching for Switch	Preview feature for extending pattern matching to switch statements.	Finalized feature, making switch more concise and expressive.
Strong Encapsulation of JDK Internals	Strengthened encapsulation by making internal JDK APIs inaccessible by default.	No changes; already present in Java 17.
UTF-8 by Default	Not available.	UTF-8 becomes the default character set for String, file I/O, and other APIs.
Simple Web Server	Not available.	Introduced a simple web server for prototyping and testing.
Virtual Threads	Not available.	Virtual threads (Preview) simplify the creation of high-throughput concurrent applications.
Structured Concurrency	Not available.	Introduced (Incubator) API to treat multiple tasks running in different threads as a single unit of work.
Record Patterns	Not available.	Record patterns (Preview) simplify destructuring record values directly in patterns.
Scoped Values	Not available.	Scoped values (Incubator) offer an alternative to thread-local variables.
Sequenced Collections	Not available.	Sequenced collections introduced to maintain a defined iteration order in collections.

https://www.geeksforgeeks.org/java/j2ee-multitier-architecture/

Records

Records

- Motivation
  - Common complaint
    - "Java is too verbose" or has "too much ceremony"
    - Especially classes that are immutable data carriers
  - Data-carrier class involves a lot of low-value, repetitive, error-prone boilerplate code
    - Constructors, accessors, equals, hashCode, toString, like the example on the right
    - The Point class is just a simple data structure
  - Developers cut corners by
    - Omitting equals leading to unexpected behavior or debugging issues
    - Using an existing inappropriate class, usually designed for some other purpose, to avoid writing low level boilerplate code

```java
class Point {
    private final int x;
    private final int y;

    Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    int x() { return x; }
    int y() { return y; }

    public boolean equals(Object o) {
        if (!(o instanceof Point)) return false;
        Point other = (Point) o;
        return other.x == x && other.y == y;
    }

    public int hashCode() {
        return Objects.hash(x, y);
    }

    public String toString() {
        return String.format("Point[x=%d, y=%d]", x, y);
    }
}
```


Image Credit: https://openjdk.org/jeps/395

Records

- Design considerations
  - Semantic goal is to model domain concepts as data structures
  - A record is a declarative declaration that represents the shape of the real-world data
  - The required boilerplate code needed can then be generated deterministically
- Records are a new type of class in the Java language
  - Pragmatically, they model plain data aggregates with less ceremony than normal classes

```
class Point {
    private final int x;
    private final int y;

    Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    int x() { return x; }
    int y() { return y; }

    public boolean equals(Object o) {
        if (!(o instanceof Point)) return false;
        Point other = (Point) o;
        return other.x == x && other.y == y;
    }

    public int hashCode() {
        return Objects.hash(x, y);
    }

    public String toString() {
        return String.format("Point[x=%d, y=%d]", x, y);
    }
}
```


Image Credit: https://openjdk.org/jeps/395

Records

- The Record declaration consists of
  - A name, optional type parameters, a header, and a body.
  - The header, which lists the components of the record class
    - The components are the variables that define the state of the data
  - For example:
    - record Point(int x, int y) { }
  - Generates the boilerplate code shown in the example

```java
public final class Point extends java.lang.Record {

    // -- Fields - private and final, one per header component
    private final int x;
    private final int y;
    // -- Canonical Constructor - one parameter per header component
    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    // -- Accessors - named after the component, NOT getX() / getY()
    public int x() { return this.x; }
    public int y() { return this.y; }

    // -- equals() - true if same type AND all components are equal
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Point)) return false;
        Point other = (Point) o;
        return this.x == other.x
            && this.y == other.y;
    }
    // -- hashCode() - derived from all components
    @Override
    public int hashCode() {
        return java.util.Objects.hash(x, y);
    }
    // -- toString() - format is ClassName[field=value, ...]
    @Override
    public String toString() {
        return "Point[x=" + x + ", y=" + y + "]";
    }
}
```

Records

- Features
  - Every record class extends java.lang.Record
    - Extending this implicit superclass means Records cannot extend any other class
  - Records are always final and cannot be subclassed
  - No getX() / getY()
    - The accessors match the component name exactly: point.x(), not point.getX()
  - equals() and hashCode() are component-based
    - Two Point instances with the same x and y are equal and have the same hash code, regardless of being different object instances

```
public final class Point extends java.lang.Record {

    // — Fields – private and final, one per header component
    private final int x;
    private final int y;
    // — Canonical Constructor – one parameter per header component
    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    // — Accessors – named after the component, NOT getX() / getY()
    public int x() { return this.x; }
    public int y() { return this.y; }

    // — equals() – true if same type AND all components are equal
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Point)) return false;
        Point other = (Point) o;
        return this.x == other.x
            && this.y == other.y;
    }
    // — hashCode() – derived from all components
    @Override
    public int hashCode() {
        return java.util.Objects.hash(x, y);
    }
    // — toString() – format is ClassName[field=value, ...]
    @Override
    public String toString() {
        return "Point[x=" + x + ", y=" + y + "]";
    }
}
```

Records

- Constructors
  - A normal class without any constructor declarations is automatically given a default constructor
    - The constructor with no arguments
  - But, a Record class without any constructor declarations is automatically given a canonical constructor
    - Derived from the arguments in the Record header
  - The canonical constructor creates final fields corresponding to the arguments in the header of the Record header

```java
public final class Point extends java.lang.Record {

    // — Fields — private and final, one per header component
    private final int x;
    private final int y;
    // — Canonical Constructor — one parameter per header component
    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    // — Accessors — named after the component, NOT getX() / getY()
    public int x() { return this.x; }
    public int y() { return this.y; }

    // — equals() — true if same type AND all components are equal
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Point)) return false;
        Point other = (Point) o;
        return this.x == other.x
            && this.y == other.y;
    }
    // — hashCode() — derived from all components
    @Override
    public int hashCode() {
        return java.util.Objects.hash(x, y);
    }
    // — toString() — format is ClassName[field=value, ...]
    @Override
    public String toString() {
        return "Point[x=" + x + ", y=" + y + "]";
    }
}
```

Records

- Constructors
  - The canonical constructor can be explicitly defined
  - This form is used to do processing of the inputs, like validity checking
  - No need to explicitly assign the parameters to the instance variables
    - This is still done implicitly
  - In the examples in the sidebar
    - The first does validity checking
    - The second normalizes the inputs

```
record Range(int lo, int hi) {
    Range {
        if (lo > hi)  // referring here to the implicit constructor parameters
            throw new IllegalArgumentException(String.format("(%d,%d)", lo, hi));
    }
}

record Rational(int num, int denom) {
    Rational {
        int gcd = gcd(num, denom);
        num /= gcd;
        denom /= gcd;
    }
}
```

Records

- Rules for record classes
  - A record class declaration cannot not have an explicit extends clause
  - A record class is implicitly final, and cannot be abstract
  - The fields derived from the Record components are final
  - A record class cannot explicitly declare instance fields, and cannot contain instance initializers
    - This ensures that the record header alone defines the state of a record value

```
// Record - this is the ONLY place to define state
record Person(String name, int age) {
    // ❌ ILLEGAL - cannot add extra instance fields
    String nickname;
    private int secretValue;
}
```

```
record Person(String name, int age) {
    // ❌ ILLEGAL - no instance initializers allowed
    { System.out.println("created!"); }

    // ✅ Use the compact constructor instead
    Person {
        if (age < 0) throw new IllegalArgumentException("Age can't be negative");
    }
}
```

Records

- Rules for Record classes
  - If you override one of the auto-generated methods, it must have the same signature as the auto-generated method
  - Any explicit implementation of accessors or the equals or hashCode methods should be careful to preserve the semantic invariants of the Record class
  - The semantic invariants of a record are:
    - Accessors return the value that was passed into the canonical constructor
    - equals() is based on all components
    - hashCode() is consistent with equals()

```
record Point(int x, int y) {

    // Legal – matches what the compiler would generate
    public int x() {
        return this.x;
    }

    // Illegal – wrong return type (long instead of int)
    public long x() {
        return this.x;
    }

    // Technically compiles but BREAKS the invariant –
    // two Points with the same x,y are no longer equal
    @Override
    public boolean equals(Object o) {
        return false;  // always false – destroys value semantics
    }

    // Also breaks invariant – accessor no longer returns
    // the value that was passed to the constructor
    @Override
    public int x() {
        return 0;  // always 0 – misleading and dangerous
    }
}
```

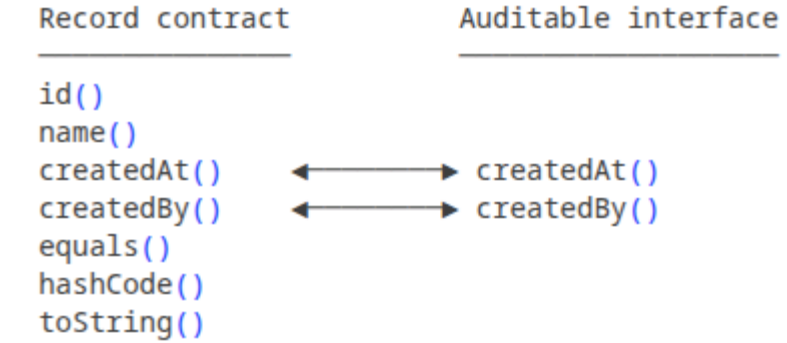
Records

- Records can implement interfaces.
  - Allows extended domain modelling
  - For example, a Record can
    - Be an Auditable entity
    - Be a Serializable payload
    - Implement a domain-specific marker interface
  - The example shown implements the Auditable interface
    - Since the methods defined in the interface correspond with record components
    - Java generates all the necessary code

```java
public interface Auditable {
    Instant createdAt();
    String createdBy();
}

// Record implementing an interface
public record AuditedEmployee(
        Long id,
        String name,
        Instant createdAt,
        String createdBy
) implements Auditable {}

// The accessor methods from the interface (createdAt(), createdBy())
// are satisfied automatically by the record components.
```



Records

- The example on the right shows how Java expands the example from the previous slide
- All of this is auto-generated
  - Continued on next slide

```java
public final class AuditedEmployee extends java.lang.Record implements Auditable {

    // ── Fields — private and final, one per header component
    private final Long    id;
    private final String  name;
    private final Instant createdAt;
    private final String  createdBy;

    // ── Canonical Constructor
    public AuditedEmployee(Long id, String name, Instant createdAt, String createdBy) {
        this.id        = id;
        this.name      = name;
        this.createdAt = createdAt;
        this.createdBy = createdBy;
    }

    // ── Accessors
    // These four methods satisfy BOTH the record contract AND the Auditable
    // interface contract simultaneously. No separate @Override needed.

    public Long id() { return this.id; }
    public String name() { return this.name; }
    public Instant createdAt() { return this.createdAt;}
    public String createdBy() {return this.createdBy;}
```

Records

- Continued from the previous slide

```java
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof AuditedEmployee)) return false;
    AuditedEmployee other = (AuditedEmployee) o;
    return java.util.Objects.equals(this.id,        other.id)
        && java.util.Objects.equals(this.name,      other.name)
        && java.util.Objects.equals(this.createdAt, other.createdAt)
        && java.util.Objects.equals(this.createdBy, other.createdBy);
}

// ── hashCode()
@Override
public int hashCode() {
    return java.util.Objects.hash(id, name, createdAt, createdBy);
}

// ── toString()
@Override
public String toString() {
    return "AuditedEmployee[" +
           "id="        + id        + ", " +
           "name="      + name      + ", " +
           "createdAt=" + createdAt + ", " +
           "createdBy=" + createdBy + "]";
}
```

Records

- Implementing Interfaces
  - Because Interfaces are contracts and do not maintain state or defined instance variables, they do not hold state
  - Implementing an interface does not undermine the data contract of the Record
  - Think of it as adding functionality to a data item, exactly as you would with a regular class in Java

```java
// Implementing Runnable - completely fine
record Task(String name, int priority) implements Runnable {
    @Override
    public void run() {
        System.out.println("Running task: " + name);
    }
}

// Multiple interfaces
record Point(int x, int y) implements Comparable<Point>, Serializable {
    @Override
    public int compareTo(Point other) {
        return Integer.compare(this.x, other.x);
    }
}
```

Record Use Cases

Use Case	Why Records Are Ideal	Example
API Request / Response DTOs	Immutable payloads; Jackson-compatible; compact	CreateEmployeeRequest, EmployeeResponse
Domain Value Objects	Enforce immutability; value equality semantics	Money(BigDecimal amount, Currency currency)
Kafka Event Payloads	Serializable, immutable, self-describing	EmployeeCreatedEvent(Long id, String name, Instant occurredAt)
Method Return Groups	Return multiple values without a dedicated class	PagedResult<T>(List<T> items, long total)
Configuration Snapshots	Read @ConfigurationProperties into a record	AppConfig(String baseUrl, int timeoutMs)
Test Fixtures / Builders	Readable, immutable test data carriers	TestEmployee.DEFAULT = new EmployeeDto(...)

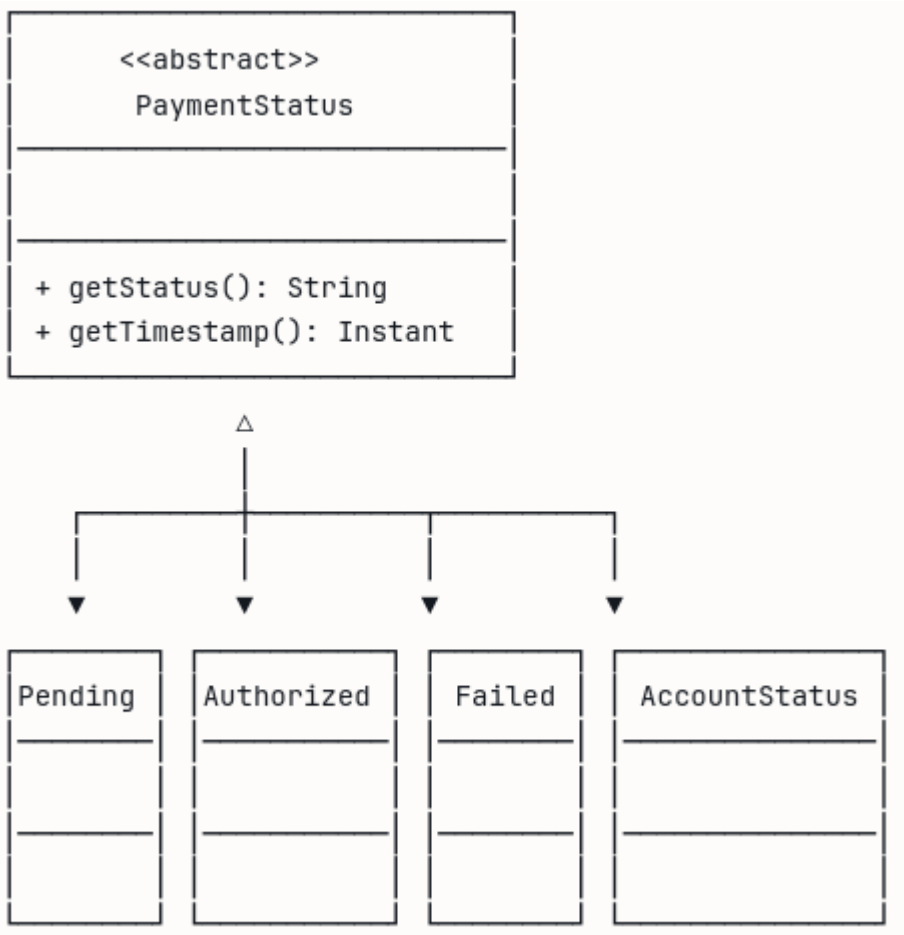
Records

- When NOT to use Records
  - When mutable state is needed
    - Records are always immutable; use a regular class with setters if mutable state is needed
  - When inheritance capabilities are needed
    - Records are final; use an abstract class hierarchy instead
  - For JPA Entities
    - Hibernate requires a no-arg constructor and mutable fields
    - Use @Entity on a regular class instead
  - When the type has significant behaviour
    - Records are data carriers, not service objects
    - A Record with 10 methods is a code smell

Sealed Classes

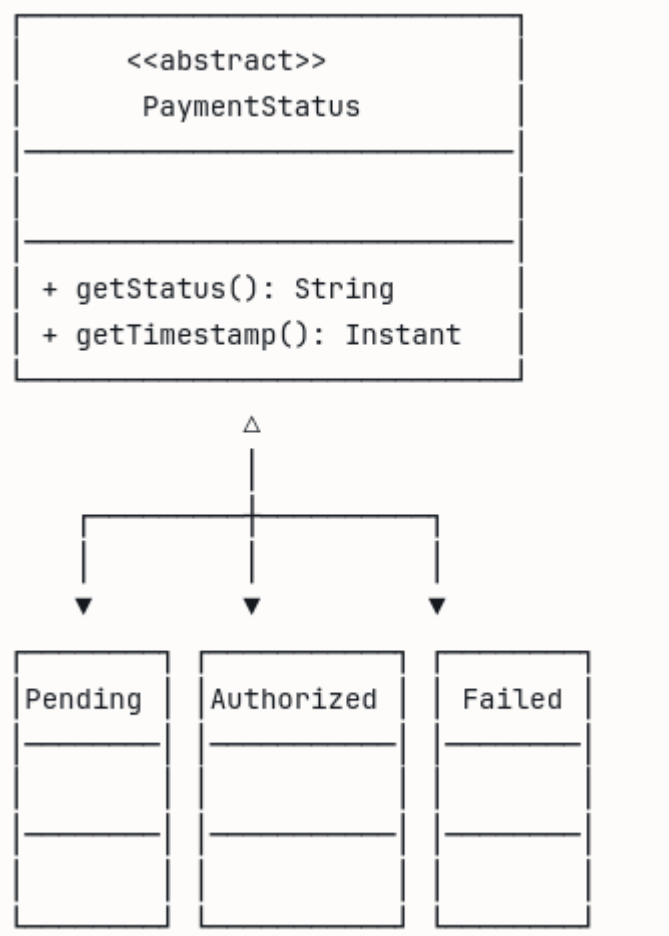
Sealed Classes

- The engineering problem
  - Open vs. closed hierarchies
  - Inheritance in Java has always been unrestricted
  - Any class can extend any non-final class in any package
  - Useful for framework extension points
  - Actively harmful for domain modelling
- For example:
  - Modeling a class hierarchy with a payment abstract base class PaymentStatus
  - Requires exactly three states: Pending, Authorised, Failed implemented as concrete subclassses
  - It is possible for a fourth class in a different package to extend PaymentStatus which could break the code
  - In the diagram, this is the class AccountStatus



Sealed Classes

- A sealed type
  - Restricts the subtypes that can extend it
  - Uses the sealed modifier
  - The permits clause identifies the types that can extend the sealed type
- Semantics
  - Represents a concept that has a fixed and known number of states and variants
  - The sealed class represents the concept, like PaymentStatus for example
  - The subclasses represent the allowed subtypes in the domain



Sealed Classes

- A common pattern
  - Records are used for the variants because each state typically carries different data
    - Authorized carries an authorization code and timestamp
    - Failed carries a failure reason and error code
    - Pending carries nothing
  - In the example, records are used to implement the permitted states
    - Since they are inside the scope of an Interface, they are by default static and public
    - They only make semantic sense inside the namespace PaymentStatus

```
sealed interface PaymentStatus permits
    PaymentStatus.Pending,
    PaymentStatus.Authorised,
    PaymentStatus.Failed {

    record Pending() implements PaymentStatus {}

    record Authorised(String authCode, Instant authorisedAt)
        implements PaymentStatus {}

    record Failed(String reason, String errorCode)
        implements PaymentStatus {}
}
```

Sealed Classes

- Constraints on its permitted subtypes
  - The sealed class and its permitted subclasses must belong to the same module
    - If declared in an unnamed module, then they must belong to the same package
    - Every permitted subclass must directly extend the sealed class
  - Subclass must use a modifier to describe how it propagates the sealing
    - May be declared final
    - May be declared sealed to allow extension but in a restricted fashion
    - May be declared non-sealed so that its part of the hierarchy reverts to being open

```
public abstract sealed class Shape
    permits Circle, Rectangle, Square, WeirdShape { }

final class  Circle extends Shape {  }

sealed class Rectangle extends Shape
    permits TransparentRectangle, FilledRectangle { }

final class TransparentRectangle extends Rectangle { }

final class FilledRectangle extends Rectangle {  }

final class Square extends Shape { }

non-sealed class WeirdShape extends Shape { }
```

Sealed Classes

Modifier on Subtype	Effect	When to Use
final	No further subclassing. Use on leaf nodes.	Most domain types — Circle, Rectangle
sealed	Restricts its own subtypes further.	Multi-level hierarchies — ApiError → ClientError, ServerError
non-sealed	Reopens the hierarchy — any class may extend.	Framework extension points where third-party extension is intentional

Enforced Completeness

- Sealed types as a safety net
  - Adding a new permitted subtype causes every switch expression over that type to fail to compile until you handle the new case
  - Transforms runtime bugs into compile-time errors across your entire codebase
  - A massive reliability improvement for domain models.
- Use sealed types for all finite-state domain concepts
  - Order status, payment state, approval

```
public sealed interface PaymentStatus permits
    PaymentStatus.Pending,
    PaymentStatus.Authorised,
    PaymentStatus.Failed,
    PaymentStatus.Refunded {       // <-- new variant added

    record Pending() implements PaymentStatus {}
    record Authorised(String authCode, Instant authorisedAt)
       implements PaymentStatus {}
    record Failed(String reason, String errorCode)
        implements PaymentStatus {}
    record Refunded(String refundId, Instant refundedAt)
        implements PaymentStatus {}
}

public String describe(PaymentStatus status) {
    return switch (status) {
        case PaymentStatus.Pending    p -> "Pending";
        case PaymentStatus.Authorised a -> "Authorised: " + a.authCode();
        case PaymentStatus.Failed     f -> "Failed: " + f.reason();
        // Refunded is not handled —
        // COMPILE ERROR: the switch expression
        // does not cover all possible input values
    };
}
```

Pattern Matching

Pattern Matching

- The check-then-cast problem
  - The legacy instanceof pattern
  - Java traditionally required a two-step idiom
    - Check the type, then cast
  - Verbose and error-prone because the check and cast can silently get out of sync
  - A common source of ClassCastException in legacy codebases

```
// BEFORE Java 16 — verbose and fragile
if (obj instanceof String) {
    String s = (String) obj;  // redundant cast
    System.out.println(s.length());
}
```

Pattern Matching

- The pattern variable
  - Combines the type check and the cast into a single expression
  - The pattern variable is automatically cast and scoped to the true branch
  - Eliminates redundant casts and removes the risk of check/cast mismatch
  - Pattern variables can be used within the same condition (e.g. && s.length() > 5)
  - Each branch remains clean, readable, and cast-free

```java
// AFTER Java 16 — concise and safe
if (obj instanceof String s) {
    System.out.println(s.length());  // s is already cast
}
```

```java
public String describe(Object obj) {
    if (obj instanceof Integer i) {
        return "Integer: " + i;
    } else if (obj instanceof String s && s.length() > 5) {
        return "Long string: " + s;  // 's' usable in the guard condition
    } else if (obj instanceof Double d) {
        return String.format("Double: %.2f", d);
    }
    return "Unknown: " + obj;
}
```

Pattern Matching

- Implementation
  - Preview feature in Java 17
  - Finalized in Java 21
- Combines type matching with switch expressions
  - Match on the type of the expression, not just equality
  - Binds the matched value to a pattern variable in a single step
  - Most powerful when combined with sealed types

```
public String formatValue(Object value) {
    return switch (value) {
        case Integer i -> "int: " + i;
        case Long l    -> "long: " + l;
        case Double d  -> String.format("double: %.3f", d);
        case String s  -> "string(" + s.length() + "): " + s;
        case null      -> "null";
        default        -> "other: " + value.getClass().getSimpleName();
    };
}
```

Guards: when Clauses

- Refining cases without nested logic
  - A when clause attaches a boolean condition to a pattern case
  - Replaces nested if-else chains inside case blocks
  - Cases are evaluated top-to-bottom
  - Place more specific guards first

```
public String classifyEmployee(Employee e) {
    return switch (e) {
        case Employee emp when emp.salary().compareTo(new BigDecimal("100000")) > 0
                               -> "Senior: " + emp.name();
        case Employee emp when emp.yearsService() > 10
                               -> "Veteran: " + emp.name();
        default                -> "Standard: " + e.name();
    };
}
```

Record Patterns

- Java 21
- Match and destructure in one step
  - Record patterns let you match a record type and extract its components inline
  - Removes the need to call accessor methods after the match
  - Works at any nesting depth: nested record patterns are supported
  - Without record patterns, you would write c.radius(), r.width(), etc. after matching

```
public double processShape(Shape shape) {
    return switch (shape) {
        case Circle(double r)              -> Math.PI * r * r;
        case Rectangle(double w, double h) -> w * h;
        case Triangle t                    -> 0.5 * t.base() * t.height();
    };
}
```

Example

- Replacing the visitor pattern
  - Sealed event hierarchies and pattern matching switch results in a clean, exhaustive dispatch
  - No default branch needed
    - The compiler guarantees all cases are handled
    - Adding a new event type causes a compile error at every unhandled switch

```
@Service
public class EmployeeEventHandler {

    public void handle(EmployeeEvent event) {
        switch (event) {
            case EmployeeCreated e    -> onCreated(e);
            case EmployeeUpdated e    -> onUpdated(e);
            case EmployeeSuspended e  -> onSuspended(e);
            case EmployeeTerminated e -> onTerminated(e);
        }
    }
}
```

Version Reference

Feature	Introduced	Finalised
`instanceof` pattern variables	Java 14 (preview)	Java 16
Switch expressions	Java 12 (preview)	Java 14
Pattern matching in switch	Java 17 (preview)	Java 21
Record patterns	Java 19 (preview)	Java 21
`when` guards in switch	Java 21	Java 21

Multi-line Strings

Multi-line Strings

- The problem
  - SQL, JSON, HTML, and XML embedded in Java required constant manual escaping
  - Newlines required explicit \n characters
  - Double quotes required \" everywhere
  - String concatenation across lines made code nearly unreadable
  - Indentation had to be managed by hand and was easily broken by refactoring
  - The code in the example is correct, but is very difficult to read

```
String sql = "SELECT e.id, e.name, d.name AS department " +
             "FROM employees e " +
             "JOIN departments d ON e.department_id = d.id " +
             "WHERE e.status = 'ACTIVE' " +
             "ORDER BY e.name";

String json = "{\n" +
              "  \"name\": \"Alice\",\n" +
              "  \"role\": \"developer\"\n" +
              "}";
```

Multi-line Strings

- Text block syntax
  - As of Java 15
  - Delimited by """ on opening and closing lines
  - Content indentation is stripped automatically by the compiler
  - Single and double quotes require no escaping inside a text block
  - A trailing newline is included automatically

```
String sql = """
        SELECT e.id, e.name, d.name AS department
        FROM employees e
        JOIN departments d ON e.department_id = d.id
        WHERE e.status = 'ACTIVE'
        ORDER BY e.name
        """;

String json = """
        {
            "name": "Alice",
            "role": "developer"
        }
        """;
```

Multi-line Strings

- Indentation rules and stripping
  - The closing delimiter controls the indent level
  - The compiler strips the common leading whitespace
    - Done by finding the minimum indent across all non-empty content lines and the position of the closing """
    - Rule of thumb: align the closing """ with the content to get clean, zero-indent output

```
// Closing delimiter at column 0 — content indent IS preserved
String s1 = """
        SELECT *
        FROM t
""";
// Result: "        SELECT *\n        FROM t\n"  (8-space indent kept)


// Closing delimiter aligned with content — indent IS stripped
String s2 = """
        SELECT *
        FROM t
        """;
// Result: "SELECT *\nFROM t\n"  (preferred style)
```

Multi-line Strings

- Inline parameterization
  - Replacies String.format() with an instance method
  - .formatted(args...) is the instance-method equivalent of String.format()
  - Chains directly onto a text block
  - Keeps the string template and its arguments visually together

```
String greeting = """
        Hello, %s!
        Your employee ID is %d.
        """.formatted(employee.name(), employee.id());
```

Companion String Methods

Method	Purpose
`.formatted(args)`	Inline parameterisation of a string template
`.stripIndent()`	Runtime version of the compiler's indent stripping
`.translateEscapes()`	Process escape sequences in externally sourced strings
`.indent(n)`	Add (`n > 0`) or remove (`n < 0`) indent on every line

```java
// .stripIndent() — strip common leading whitespace at runtime
// Useful when text arrives from an external source (database, config)
String stripped = fetchTemplateFromDatabase().stripIndent();


// .translateEscapes() — process \n, \t etc. in strings from external sources
String withEscapes  = "line1\\nline2";
String translated   = withEscapes.translateEscapes();
// Result: "line1\nline2"


// .indent(n) — add or remove leading indent programmatically
String indented = "SELECT *\nFROM t".indent(4);
// Result: "    SELECT *\n    FROM t\n"
```

Virtual Threads

Virtual Threads

- Only available in Java 21
  - Referred to as Project Loom
- A virtual thread
  - Exists in the JVM's world but does not have a 1-to-1 mapping with an OS thread
  - The JVM itself acts as a scheduler, managing potentially millions of virtual threads on top of a small, fixed pool of real OS threads (called carrier threads)
- The JVM can observe something a CPU cannot
  - It knows exactly when a virtual thread is waiting rather than working
    - When a virtual thread blocks, for example whil ewaiting for a database response, a network socket, a file read, it has nothing useful to do
    - The JVM detects this and parks it, freeing its carrier thread to pick up another virtual thread that does have work to do

Virtual Threads

- Traditional OS threads conflate two distinct concept
  - A unit of concurrency
    - Something the program wants to do simultaneously with other things
  - A scheduling resource
    - A slice of CPU time granted by the OS
  - Fine when threads are mostly computing
  - It breaks down badly when threads spend most of their time waiting
    - Which is exactly what happens in I/O-heavy enterprise workloads (web servers, database clients, message consumers)
    - You end up reserving an expensive OS resource for a thread that is doing nothing
  - Virtual threads decouple these two concepts
    - OS scheduling resources are only consumed when there is actual work to execute

Virtual Threads

- The JVM
  - Maintains a small pool of carrier thread, typically one per CPU core
    - Virtual threads are mounted onto a carrier thread to run, and unmounted when they block
    - This is purely a JVM-level operation:

```
Virtual Thread A  ⟶ runs on Carrier Thread 1
Virtual Thread A blocks on DB call
  ↳⟶ JVM parks A, unmounts it from Carrier Thread 1
Virtual Thread B  ⟶ now runs on Carrier Thread 1
...DB response arrives
  ↳⟶ JVM unparks A, schedules it to resume on a carrier thread
```

  - No OS context switch occurs during the park/unpark cycle
    - This is the core efficiency gain the expensive part of traditional thread switching is bypassed entirely

Use Cases

- Well-suited for
  - High-concurrency I/O workloads
    - For example, web servers handling thousands of simultaneous requests, each doing DB or API calls
  - Thread-per-request models
    - Each request gets its own thread
    - Blocking is fine because parking is cheap
  - Simplifying reactive code
    - Code written in a straightforward blocking style can achieve reactive-level throughput without the complexity

- Poorly suited for
  - CPU-bound workloads
    - If a thread is constantly computing (image processing, cryptography, ML inference)
    - It never parks, and virtual threads offer no advantage over platform threads.
    - The bottleneck is CPU time, not thread count
  - Anything using synchronized blocks heavily
    - A virtual thread that blocks inside a synchronized block pins to its carrier thread rather than parking
    - This negates much of the benefit of virtual threads

Virtual Threads - Versions

	Java 17	Java 21
Virtual threads	Not available	GA / Standard (JEP 444)
Max practical threads	~2,000–5,000	Millions
Blocking I/O cost	Wastes an OS thread	Parks virtual thread; OS thread freed
Thread pool tuning	Critical	Not needed
Memory per thread	~1 MB stack	~few KB
High concurrency approach	Reactive (WebFlux) required	Blocking code scales natively
Spring Boot support	N/A	One config line in Boot 3.2+

Questions?